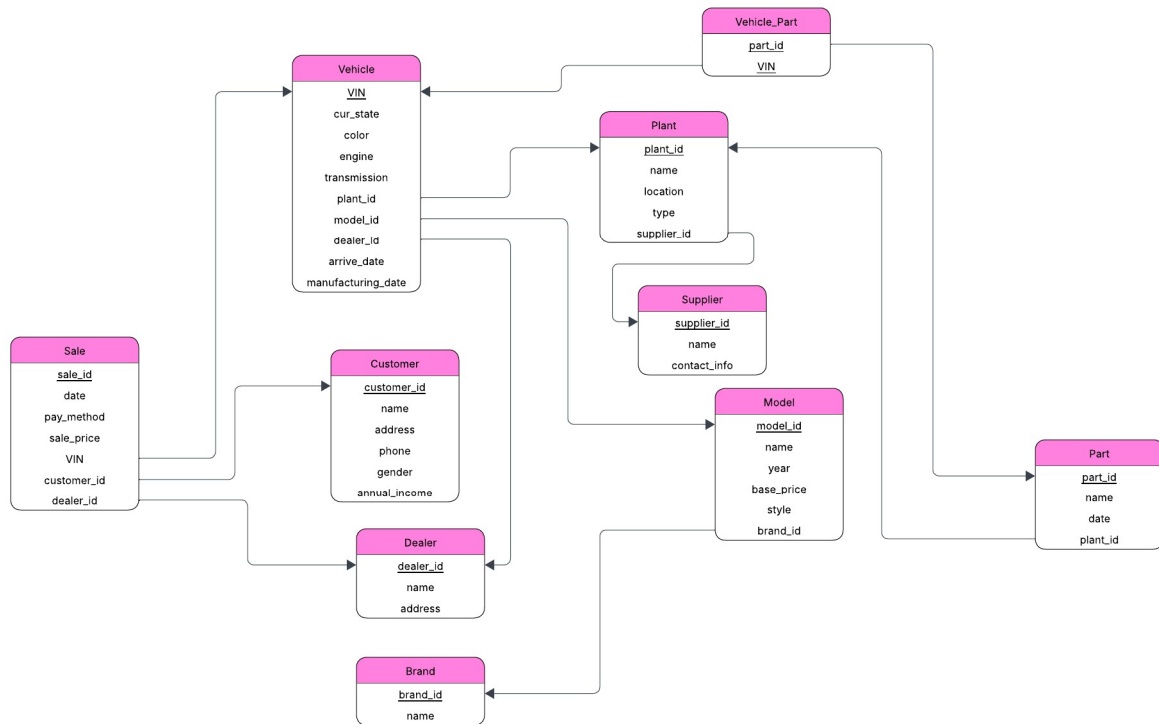


-Logical Schema Design



Brand (**brand_id** (PK), name)

Supplier (**supplier_id** (PK), name, contact_info)

Dealer (**dealer_id** (PK), name, address)

Customer (**customer_id** (PK), name, address, phone, gender, annual_income)

Model (**model_id** (PK), name, year, base_price, style, *brand_id* (FK to Brand))

Plant (**plant_id** (PK), name, location, type, *supplier_id* (FK to Supplier))

Part (**part_id** (PK), name, date, *plant_id* (FK to Plant))

Vehicle (**VIN** (PK), cur_state, color, engine, transmission, manufacturing_date, *model_id* (FK to Model), *plant_id* (FK to Plant), *dealer_id* (FK to Dealer))

Sale (**sale_id** (PK), date, pay_method, sale_price, *VIN* (FK to Vehicle), *customer_id* (FK to Customer), *dealer_id* (FK to Dealer))

Vehicle_Part (**VIN** (PK/FK to Vehicle), **part_id** (PK/FK to Part))

로 정리할 수 있다.

Brand, Supplier, Dealer, Customer과 같은 entity들은 weak entity가 아니므로 각각의 id들을 primary key로 하여 구성하였다. Brand의 경우에는 여러 개의 Model을 가질 수 있으므로 Model에 brand_id를 포함시켰고, Customer가 여러 대의 차량을 구매할 수 있으므로 Sale에 customer_id를 포함시킨다. 마찬가지로 Plant의 경우 여러 부품 생산이 가능하므로 Part에 plant_id를 추가하고, Supplier는 여러 Plant 소유가 가능하므로 Plant에 supplier_id를 추가한다. Dealer도 여러 판매가 가능하므로 Sale에 dealer_id를 포함시키고, 차량도 여러 대 가능하므로 VIN을 추가한다.

Vehicle에는 여러 Part가 가능하고 Part도 여러 Vehicle에 쓰일 수 있으므로 Vehicle_Part를 추가하여 VIN과 part_id를 가지고 있는 entity를 생성한다.

-Normalization Analysis

-Customer

FD : customer_id -> {name, address, phone, gender, annual_income}

customer_id만 후보키이고 superkey이므로 BCNF를 만족한다.

-Model

FD : model_id -> {name, year, base_price, style, brand_id}

model_id는 이름, year, base_price 등을 결정하고 외래키인 brand_id는 다른 속성 결정에 관여하지 않으므로 BCNF 만족

-Vehicle

FD : VIN -> {cur_state, color, engine, transmission, manufacturing_date, model_id, plant_id, dealer_id}

VIN 이외에 다른 속성을 결정하는 집합은 없으므로 BCNF를 만족한다.

-Sale

FD : sale_id -> {date, pay_method, sale_price, VIN, customer_id, dealer_id}

VIN -> sale_id

가 성립하고 sale_id, VIN 모두 후보키이므로 BCNF를 만족한다.

-Vehicle_Part

{VIN, part_id}

Primary key들로만 구성된 entity로 functional dependency가 존재하지 않으므로 BCNF 만족

-Brand

FD : brand_id -> {name}

brand_id가 후보키이므로 BCNF 만족

-Supplier

FD : supplier_id -> {name, contact_info}

오직 supplier_id에 의해서만 결정되며 이 것이 superkey이므로 BCNF

-Dealer

FD : dealer_id -> {name, address}

dealer_id가 후보키이므로 BCNF 만족

-Plant

FD : plant_id -> {name, location, type, supplier_id}

모든 정보가 plant_id에 의해 결정되고 supplier_id는 다른 속성 결정하지 않으므로 plant_id가 후보키가 되어 BCNF 만족

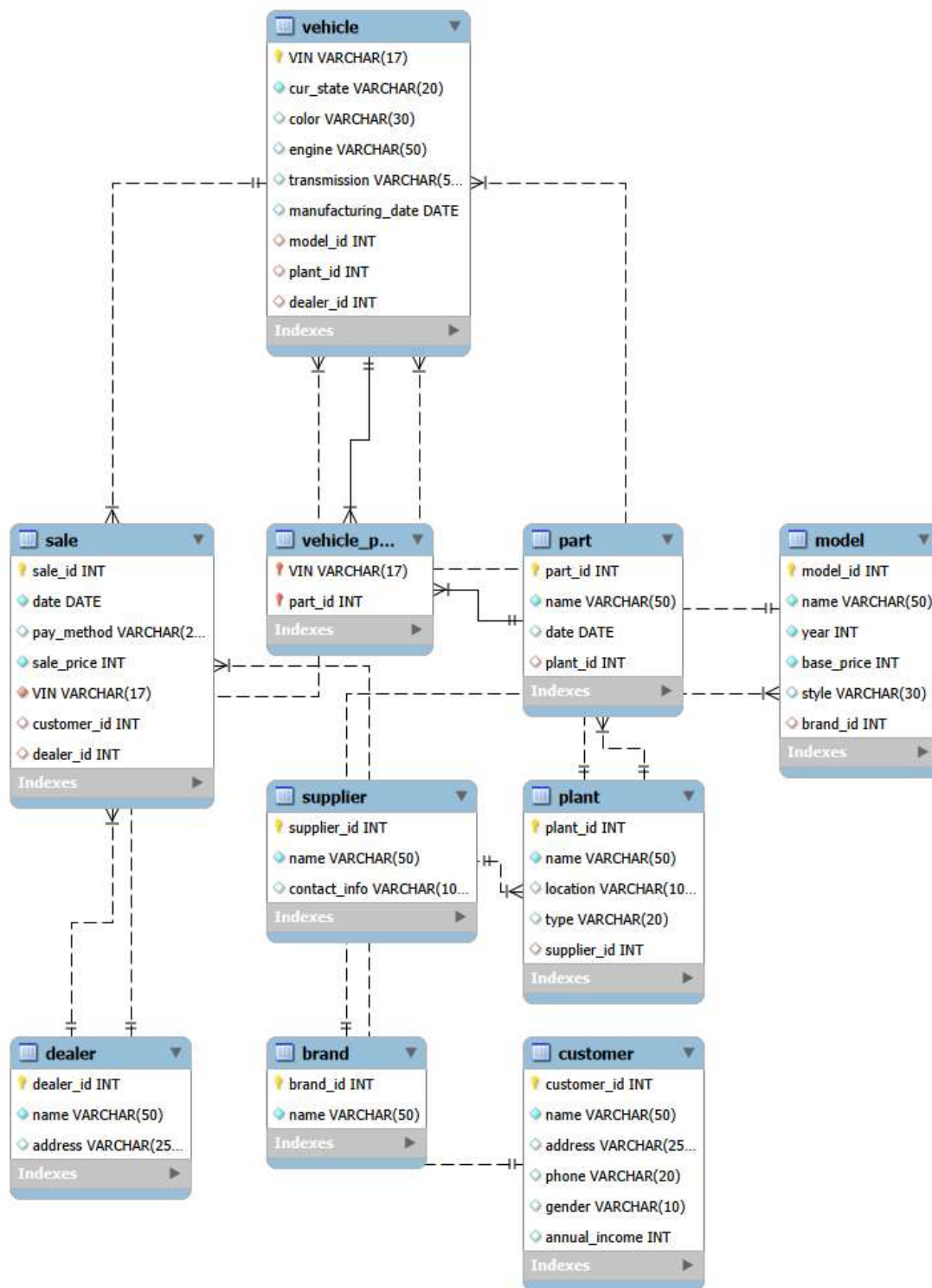
-Part

FD : part_id -> {name, date, plant_id}

속성들이 part_id에 의해 결정되고 후보키이므로 BCNF 만족

전반적으로 relation별로 superkey가 존재하여 BCNF에 위배되지 않는 구조이므로 구성한 schema가 BCNF를 만족하는 것을 확인할 수 있다.

-Physical Implementation



데이터 타입의 경우 INT와 VARCHAR를 선택하여 사용하였고, date와 같이 날짜를 사용하는 경우에 DATE type을 사용하였다.

VIN을 제외한 나머지 primary key들에는 AUTO_INCREMENT를 적용하여 primary key의 충돌을 방지하였고, name, year, base_price 등과 같이 비어있으면 안 되는 속성의 경우에는 NOT NULL로 제약조건을 명시했다. 또한 Sale에서 VIN은 UNIQUE 처리를 함으로 하

나의 VIN은 하나의 판매 기록만 가지도록 제한하고, Sale->Vehicle, Vehicle_Part->Vehicle, Part에서는 ON DELETE CASCADE로 제거 시 함께 제거되도록 하였다. Vehicle->Model에서는 ON DELETE RESTRICT로 Vechicle이 존재 시에 Model을 지우지 않도록 하였고, 이 외의 관계에서는 ON DELETE SET NULL로 Dealer 등이 삭제되더라도 Sale이 사라지지 않도록 하는 등의 조건처리를 하였다. 또한 대용량 데이터 환경에서의 쿼리 성능 최적화를 위해 다수의 조인과 조건절에 사용되는 Sale의 date 속성과 Supplier의 name 속성 등에 인덱스(INDEX)를 생성하였다. 이를 통해 기간 조회 및 특정 공급사 검색 쿼리 실행 시 전체 테이블 스캔을 방지하고, 효율적인 인덱스 스캔이 이루어지도록 물리적 스키마를 설계하였다.

데이터의 경우 테이블당 최소 10개의 tuple을 만족하기 위해서 10개의 relation에 대해 105개의 tuple을 넣었다. 또한 데이터들은 7개의 쿼리를 테스트 시에 누락되지 않도록 구성하였는데 자세한 부분은 아래와 같다.

```
USE car_company;

INSERT INTO Brand (name) VALUES
('Hyundai'), ('Kia'), ('Genesis'), ('Toyota'), ('Honda'),
('Ford'), ('Chevrolet'), ('BMW'), ('Mercedes-Benz'), ('Audi');

INSERT INTO Dealer (name, address) VALUES
('Seoul Central Auto', '123 Gangnam-daero, Seoul'),
('Busan Motors', '456 Haeundae-ro, Busan'),
('Incheon Car Dealership', '789 Songdo-dong, Incheon'),
('Daegu Auto Hub', '101 Dongdaegu-ro, Daegu'),
('Daejeon Mobility', '202 Dunsan-daero, Daejeon'),
('Gwangju Motors', '303 Sangmu-daero, Gwangju'),
('Ulsan Premium Cars', '404 Samsan-ro, Ulsan'),
('Suwon Car Center', '505 Maesan-ro, Suwon'),
('Changwon Dealership', '606 Jungang-daero, Changwon'),
('Goyang Auto Square', '707 Kintex-ro, Goyang');

INSERT INTO Supplier (name, contact_info) VALUES
```

```

INSERT INTO Supplier (name, contact_info) VALUES
('Hankook Tire', 'contact@hankook.com'),
('Kumho Tire', 'info@kumho.com'),
('Bosch Korea', 'info@bosch.co.kr'),
('Mobis', 'parts@mobis.com'),
('Denso', 'sales@denso.com'),
('Magna International', 'contact@magna.com'),
('ZF Friedrichshafen', 'support@zf.com'),
('Continental', 'info@continental.com'),
('Aisin', 'parts@aisin.com'),
('Valeo', 'contact@valeo.com');

INSERT INTO Customer (name, address, phone, gender, annual_income) VALUES
('Alice Smith', '101 Apple St, Seoul', '010-1111-2222', 'Female', 85000),
('Bob Johnson', '202 Banana Ave, Busan', '010-3333-4444', 'Male', 45000),
('Charlie Kim', '303 Cherry Blvd, Incheon', '010-5555-6666', 'Male', 120000),
('Diana Lee', '404 Date St, Daegu', '010-7777-8888', 'Female', 62000),
('Evan Park', '505 Fig Ave, Daejeon', '010-9999-0000', 'Male', 150000),
('Fiona Choi', '606 Grape Blvd, Gwangju', '010-1234-5678', 'Female', 38000),
('George Jung', '707 Hazel St, Ulsan', '010-8765-4321', 'Male', 95000),
('Hannah Kang', '808 Ivy Ave, Suwon', '010-2345-6789', 'Female', 110000),
('Ian Cho', '909 Jack St, Changwon', '010-9876-5432', 'Male', 72000),
('Julia Yoon', '1010 Kiwi Blvd, Goyang', '010-3456-7890', 'Female', 54000);

```

우선 다른 entity를 참조하지 않는 Brand, Dealer, Supplier, Customer를 우선 넣고

```

INSERT INTO Model (name, year, base_price, style, brand_id) VALUES
('Sonata', 2025, 30000, 'Sedan', 1),
('Sorento', 2025, 40000, 'SUV', 2),
('G80', 2026, 60000, 'Sedan', 3),
('Camry', 2024, 32000, 'Sedan', 4),
('CR-V', 2025, 35000, 'SUV', 5),
('Mustang', 2025, 55000, 'Convertible', 6),
('Silverado', 2024, 45000, 'Truck', 7),
('Z4', 2026, 65000, 'Convertible', 8),
('E-Class', 2025, 70000, 'Sedan', 9),
('Q7', 2026, 75000, 'SUV', 10);

INSERT INTO Plant (name, location, type, supplier_id) VALUES
('Ulsan Assembly', 'Ulsan', 'final assembly', NULL),
('Hwaseong Assembly', 'Hwaseong', 'final assembly', NULL),
('Gwangmyeong Assembly', 'Gwangmyeong', 'final assembly', NULL),
('Kentucky Plant', 'USA', 'final assembly', NULL),
('Munich Plant', 'Germany', 'final assembly', NULL),
('Mobis Asan Center', 'Asan', 'parts', 4),
('ZF Transmission Factory', 'Germany', 'parts', 7),
('Bosch Electronics', 'Daejeon', 'parts', 3),
('Hankook Geumsan', 'Geumsan', 'parts', 1),
('Denso Changwon', 'Changwon', 'parts', 5);

```



```
INSERT INTO Part (name, date, plant_id) VALUES
('V6 Engine', '2024-01-15', 6),
('18inch Alloy Wheel', '2024-02-20', 9),
('LED Headlight', '2024-03-10', 8),
('8-speed Transmission', '2024-05-01', 7),
('6-speed Transmission', '2024-05-15', 7),
('Brake Pad', '2024-06-20', 8),
('Airbag Module', '2024-07-11', 6),
('AC Compressor', '2024-08-30', 10),
('Navigation System', '2024-09-15', 6),
('EV Battery Pack', '2024-10-05', 8);
```

Model, Plant, Part를 삽입한다.

```
INSERT INTO Vehicle (VIN, cur_state, color, engine, transmission, manufacturing_date, model_id, plant_id, dealer_id) VALUES
('VIN00000000000001', 'sold', 'Black', '2.5L', '8-speed', '2024-01-20', 1, 1, 1),
('VIN00000000000002', 'sold', 'White', '2.0L', '8-speed', '2024-06-10', 8, 5, 2),
('VIN00000000000003', 'sold', 'Silver', '3.5L', '8-speed', '2024-06-15', 6, 4, 3),
('VIN00000000000004', 'sold', 'Red', '1.6L', '6-speed', '2024-07-01', 2, 2, 4),
('VIN00000000000005', 'sold', 'Blue', '3.0L', '8-speed', '2025-02-10', 3, 1, 5),
('VIN00000000000006', 'sold', 'Black', '2.5L', 'eCVT', '2025-03-20', 4, 4, 6),
('VIN00000000000007', 'sold', 'White', '2.0L', 'CVT', '2025-08-15', 5, 4, 7),
('VIN00000000000008', 'sold', 'Gray', '3.0L', '8-speed', '2026-01-10', 9, 5, 8),
('VIN00000000000009', 'sold', 'Black', '4.0L', '8-speed', '2026-02-05', 10, 5, 9),
('VIN00000000000010', 'sold', 'Yellow', '2.5L', '8-speed', '2026-03-12', 1, 1, 10),
('VIN00000000000011', 'stock', 'White', '2.5L', '8-speed', '2025-01-10', 1, 1, 1),
('VIN00000000000012', 'stock', 'Black', '3.5L', '8-speed', '2026-05-01', 3, 1, 2),
('VIN00000000000013', 'stock', 'Red', '2.0L', '8-speed', '2026-05-15', 6, 4, 3),
('VIN00000000000014', 'in production', 'Blue', '3.0L', '8-speed', '2026-06-01', 9, 5, NULL),
('VIN00000000000015', 'in production', 'Silver', '1.6L', '6-speed', '2026-06-10', 2, 2, NULL);

INSERT INTO Sale (date, pay_method, sale_price, VIN, customer_id, dealer_id) VALUES
('2024-02-15', 'Credit Card', 29000, 'VIN00000000000001', 1, 1),
('2024-07-20', 'Bank Transfer', 64000, 'VIN00000000000002', 3, 2),
('2024-08-05', 'Finance', 54000, 'VIN00000000000003', 5, 3),
('2024-09-10', 'Credit Card', 39000, 'VIN00000000000004', 2, 4),
('2025-03-01', 'Bank Transfer', 59000, 'VIN00000000000005', 7, 5),
('2025-04-15', 'Finance', 31000, 'VIN00000000000006', 4, 6),
('2025-09-20', 'Credit Card', 34000, 'VIN00000000000007', 6, 7),
('2026-02-15', 'Bank Transfer', 69000, 'VIN00000000000008', 8, 8),
('2026-03-10', 'Finance', 74000, 'VIN00000000000009', 10, 9),
('2026-04-05', 'Credit Card', 30000, 'VIN00000000000010', 9, 10);
```



```
INSERT INTO Vehicle_Part (VIN, part_id) VALUES
('VIN0000000000000001', 1), ('VIN0000000000000001', 2),
('VIN0000000000000002', 4),
('VIN0000000000000003', 4),
('VIN0000000000000004', 5),
('VIN0000000000000005', 1), ('VIN0000000000000005', 3),
('VIN0000000000000006', 6), ('VIN0000000000000007', 7),
('VIN0000000000000008', 8), ('VIN0000000000000009', 9),
('VIN0000000000000010', 2), ('VIN0000000000000011', 1),
('VIN0000000000000012', 3), ('VIN0000000000000013', 4);
```

Vehicle, Sale, Vehicle_Part는 이전에 삽입한 데이터에 맞게 id를 넣어서 데이터를 구성한다.

쿼리를 적절히 검증하기 위해서

```
('8-speed Transmission', '2024-05-01', 7),
('6-speed Transmission', '2024-05-15', 7),
```

위의 두 Part는

```
('ZF Transmission Factory', 'Germany', 'parts', 7),
```

해당 Plant의

```
('ZF Friedrichshafen', 'support@zf.com'),
```

위의 Supplier로 구성하고 Vehicle에 Vehicle_Part에서 mapping하여 결함이 있는 부품을 검증할 수 있도록 하였고,

cur_state는 sold, stock, in production 3가지 구분으로 in production의 경우 dealer가 NULL로 넣었고 VIN의 11번 차량의 경우 오래 팔리지 않은 데이터로 삽입하였다.

또한 날짜들을 고르게 분포시켜 주어진 쿼리에서 고르게 나타나도록 구성하였다.

-Application Development

```

int main() {
    MYSQL *conn;
    MYSQL_RES *res;
    MYSQL_ROW row;

    // 1. MySQL 핸들 초기화
    conn = mysql_init(NULL);
    if (conn == NULL) {
        std::cerr << "mysql_init() failed" << std::endl;
        return 1;
    }

    std::cout << "Connecting to database..." << std::endl;

    // 2. 데이터베이스 연결 (ODBC DSN 방식 -> 직접 연결 방식)
    if (mysql_real_connect(conn, "localhost", "admin", "1234", "car_company", 3306, NULL, 0) == NULL) {
        std::cerr << "Database Connection Failed! Please check your credentials." << std::endl;
        ShowMySQLError(conn);
        mysql_close(conn);
        return 1;
    }
}

```

데이터베이스 연결 파트로 admin 사용자, 비밀번호는 1234를 사용하였다. mysql.h를 사용하여 연결하였고, mysql_real_connect를 통해서 통신을 수행한다.

```

// 한글 등 다국어 처리를 위해 UTF-8 설정
mysql_set_character_set(conn, "utf8mb4");
std::cout << "Successfully connected to the Car Company Database!\n" << std::endl;

while (true) {
    // Main Menu UI
    std::cout << "\n==== Car Company Analytics System =====" << std::endl;
    std::cout << "1. Sales Trends (Past 3 Years, Demographic)" << std::endl;
    std::cout << "2. Defective Part Tracking (Secure Parameterized)" << std::endl;
    std::cout << "3. Top 2 Brands by Revenue (Past Year)" << std::endl;
    std::cout << "4. Top 2 Brands by Unit Sales (Past Year)" << std::endl;
    std::cout << "5. Seasonal Sales Patterns (Secure Parameterized)" << std::endl;
    std::cout << "6. Dealer Inventory Efficiency" << std::endl;
    std::cout << "7. Supplier Coverage Analysis" << std::endl;
    std::cout << "8. Exit Application" << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << "Select menu (1-8): ";

    int choice;
    std::cin >> choice;
}

```

쿼리 선택의 경우에 7가지의 쿼리를 숫자 입력으로 선택하도록 하였고

```

if (std::cin.fail()) {
    std::cin.clear();
    std::cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
    std::cout << "Invalid input. Please enter a number.\n" << std::endl;
    continue;
}

```

잘못된 input에 대한 예외처리를 하였다.

```

else if (choice == 5) {
    // [Query 5] 사용자 입력을 통한 계절별 판매 패턴 분석
    std::cout << "\n[Seasonal Sales Patterns]" << std::endl;
    std::cout << "Enter the vehicle style to search (e.g., Sedan, SUV, Convertible): ";

    // 입력 버퍼 정리
    std::cin.ignore((std::numeric_limits<std::streamsize>::max)(), '\n');
    char styleStr[30];
    std::cin.getline(styleStr, sizeof(styleStr));
}

```

추가 입력을 받는 query 5를 보면 위처럼 입력을 받고

```

MYSQL_STMT *stmt = mysql_stmt_init(conn);
if (!stmt) continue;

if (mysql_stmt_prepare(stmt, query, strlen(query))) {
    ShowStmtError(stmt);
    mysql_stmt_close(stmt);
    continue;
}

```

stmt를 사용하여 parameterized query를 사용하였고

```

// 출력 데이터 바인딩
MYSQL_BIND out_bind[2];
memset(out_bind, 0, sizeof(out_bind));
int bestMonth = 0, totalSales = 0;
out_bind[0].buffer_type = MYSQL_TYPE_LONG; out_bind[0].buffer = &bestMonth;
out_bind[1].buffer_type = MYSQL_TYPE_LONG; out_bind[1].buffer = &totalSales;

mysql_stmt_bind_result(stmt, out_bind);

if (mysql_stmt_execute(stmt)) {
    ShowStmtError(stmt);
} else {
    if (mysql_stmt_fetch(stmt) == 0) {
        std::printf("The best-selling month for '%s' is Month %d with %d units sold.\n",
                    styleStr, bestMonth, totalSales);
    } else {
        std::cout << "No sales data found for this style." << std::endl;
    }
}
mysql_stmt_close(stmt);

```

위처럼 실행하도록 구현한 점을 확인할 수 있다.

-Testing and Validation

[query 1]

```
const char* queries[6] = {  
    // 1. Year & Gender  
    "SELECT B.name, YEAR(S.date), C.gender, COUNT(S.sale_id), SUM(S.sale_price) "  
    "FROM Sale S JOIN Vehicle V ON S.VIN = V.VIN JOIN Model M ON V.model_id = M.model_id "  
    "JOIN Brand B ON M.brand_id = B.brand_id JOIN Customer C ON S.customer_id = C.customer_id "  
    "WHERE S.date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR) "  
    "GROUP BY B.name, YEAR(S.date), C.gender ORDER BY YEAR(S.date) DESC, B.name",  
  
    // 2. Year & Income  
    "SELECT B.name, YEAR(S.date), "  
    "CASE WHEN C.annual_income < 50000 THEN 'Low' WHEN C.annual_income BETWEEN 50000 AND 100000 THEN 'Medium' ELSE 'High' END, "  
    "COUNT(S.sale_id), SUM(S.sale_price) "  
    "FROM Sale S JOIN Vehicle V ON S.VIN = V.VIN JOIN Model M ON V.model_id = M.model_id "  
    "JOIN Brand B ON M.brand_id = B.brand_id JOIN Customer C ON S.customer_id = C.customer_id "  
    "WHERE S.date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR) "  
    "GROUP BY B.name, YEAR(S.date), CASE WHEN C.annual_income < 50000 THEN 'Low' WHEN C.annual_income BETWEEN 50000 AND 100000 THEN 'Medium' ELSE 'High' END "  
    "ORDER BY YEAR(S.date) DESC, B.name",  
};
```

위와 같이 Year, Month, Week별로 Gender에 따라, Income에 따라 나눈 테이블 6개를 출력한다.

Select menu (1-8): 1

[1/6] Sales Trends: Year & Gender

Brand	Year	Gender	Units	Revenue
Audi	2026	Female	1	74000
Hyundai	2026	Male	1	30000
Mercedes-Benz	2026	Female	1	69000
Genesis	2025	Male	1	59000
Honda	2025	Female	1	34000
Toyota	2025	Female	1	31000
BMW	2024	Male	1	64000
Ford	2024	Male	1	54000
Hyundai	2024	Female	1	29000
Kia	2024	Male	1	39000

[2/6] Sales Trends: Year & Income Range

Brand	Year	Income	Units	Revenue
Audi	2026	Medium	1	74000
Hyundai	2026	Medium	1	30000
Mercedes-Benz	2026	High	1	69000
Genesis	2025	Medium	1	59000
Honda	2025	Low	1	34000
Toyota	2025	Medium	1	31000
BMW	2024	High	1	64000
Ford	2024	High	1	54000
Hyundai	2024	Medium	1	29000
Kia	2024	Low	1	39000

[3/6] Sales Trends: Month & Gender

Brand	Month	Gender	Units	Revenue

Hyundai	2026-04	Male	1	30000
Audi	2026-03	Female	1	74000
Mercedes-Benz	2026-02	Female	1	69000
Honda	2025-09	Female	1	34000
Toyota	2025-04	Female	1	31000
Genesis	2025-03	Male	1	59000
Kia	2024-09	Male	1	39000
Ford	2024-08	Male	1	54000
BMW	2024-07	Male	1	64000
Hyundai	2024-02	Female	1	29000

[4/6] Sales Trends: Month & Income Range

Brand	Month	Income	Units	Revenue

Hyundai	2026-04	Medium	1	30000
Audi	2026-03	Medium	1	74000
Mercedes-Benz	2026-02	High	1	69000
Honda	2025-09	Low	1	34000
Toyota	2025-04	Medium	1	31000
Genesis	2025-03	Medium	1	59000
Kia	2024-09	Low	1	39000
Ford	2024-08	High	1	54000
BMW	2024-07	High	1	64000
Hyundai	2024-02	Medium	1	29000

[5/6] Sales Trends: Week & Gender

Brand	Week	Gender	Units	Revenue

Hyundai	2026-W14	Male	1	30000
Audi	2026-W10	Female	1	74000
Mercedes-Benz	2026-W07	Female	1	69000
Honda	2025-W37	Female	1	34000
Toyota	2025-W15	Female	1	31000
Genesis	2025-W08	Male	1	59000
Kia	2024-W36	Male	1	39000
Ford	2024-W31	Male	1	54000
BMW	2024-W28	Male	1	64000
Hyundai	2024-W06	Female	1	29000

[6/6] Sales Trends: Week & Income Range				
Brand	Week	Income	Units	Revenue
Hyundai	2026-W14	Medium	1	30000
Audi	2026-W10	Medium	1	74000
Mercedes-Benz	2026-W07	High	1	69000
Honda	2025-W37	Low	1	34000
Toyota	2025-W15	Medium	1	31000
Genesis	2025-W08	Medium	1	59000
Kia	2024-W36	Low	1	39000
Ford	2024-W31	High	1	54000
BMW	2024-W28	High	1	64000
Hyundai	2024-W06	Medium	1	29000

다차원 집계 시 단일 GROUP BY로 인해 발생하는 데이터 파편화 현상을 방지하고자, 애플리케이션 계층에서 총 6개의 세분화된 쿼리(연/성별, 연/소득구간, 월/성별, 월/소득구간, 주/성별, 주/소득구간)를 데이터베이스 엔진에 순차적으로 요청하도록 설계하였다. 6개의 쿼리가 실행될 때 MySQL 옵티마이저는 WHERE 절의 DATE_SUB 조건을 분석하여 Sale 테이블의 date 속성에 생성된 인덱스를 타게 함으로써 최근 3년 치 트랜잭션만 인덱스 레인지 스캔으로 빠르게 추출한다. 이후 필터링된 결과 집합을 드라이빙 데이터(Driving Data)로 삼아 Vehicle, Model, Brand, Customer과 PK-FK 관계를 기반으로 조인을 수행한다. 조인된 가상 릴레이션 위에서는 DATE_FORMAT(월 단위 변환), CONCAT, CASE 등의 내장 함수가 평가된다. 최종적으로 옵티마이저는 6개의 각기 다른 GROUP BY 조건 및 COUNT, SUM 연산을 처리하기 위해 내부 임시 테이블을 생성하며, 집계된 결과를 ORDER BY에 따라 정렬하여 6개의 독립적인 결과 셋을 도출하는 구조로 동작한다.

[query 2]

```
const char *query =
    "SELECT V.VIN, C.name, C.phone, P.name, CAST(P.date AS CHAR) FROM Vehicle_Part VP "
    "JOIN Part P ON VP.part_id = P.part_id JOIN Plant Pl ON P.plant_id = Pl.plant_id "
    "JOIN Supplier Su ON Pl.supplier_id = Su.supplier_id "
    "JOIN Vehicle V ON VP.VIN = V.VIN JOIN Sale S ON V.VIN = S.VIN "
    "JOIN Customer C ON S.customer_id = C.customer_id "
    "WHERE Su.name = ? AND P.date BETWEEN ? AND ?";
```


Select menu (1-8): 2

[Defective Part Tracking]

Enter Supplier Name (e.g., ZF Friedrichshafen): ZF Friedrichshafen

Enter Start Date (YYYY-MM-DD, e.g., 2024-05-01): 2024-05-01

Enter End Date (YYYY-MM-DD, e.g., 2024-05-31): 2024-05-31

Tracking Defective Parts from 'ZF Friedrichshafen' (2024-05-01 to 2024-05-31)...

Affected VIN	Customer Name	Phone	Part Name	Date
VIN00000000000002	Charlie Kim	010-5555-6666	8-speed Transmission	2024-05-01
VIN00000000000003	Evan Park	010-9999-0000	8-speed Transmission	2024-05-01
VIN00000000000004	Bob Johnson	010-3333-4444	6-speed Transmission	2024-05-15

사용자가 입력한 공급사 이름과 부품 생산 날짜 구간 조건이 파라미터 바인딩을 통해 프리페어드 스테이트먼트 형태로 안전하게 전달됩니다. 엔진은 Supplier의 name 인덱스 및 Part 테이블의 날짜 조건을 바탕으로 타겟 레코드를 효율적으로 Selection한다. 이렇게 식별된 소량의 공급사/부품 레코드를 Driving Table로 취급하여 Plant, Vehicle_Part, Vehicle, Sale, Customer 순으로 PK-FK 라인을 따라 중첩 루프 조인을 수행한다. 이 구조는 대용량 차량 데이터 중 결함 부품이 장착된 타겟 차량과 최종 구매자의 정보만을 디스크 I/O를 최소화하여 찾아내도록 최적화되어 있다.

[query 3]

```
const char *query =
    "SELECT B.name, SUM(S.sale_price) AS Total_Revenue FROM Sale S "
    "JOIN Vehicle V ON S.VIN = V.VIN JOIN Model M ON V.model_id = M.model_id "
    "JOIN Brand B ON M.brand_id = B.brand_id "
    "WHERE S.date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR) "
    "GROUP BY B.brand_id, B.name ORDER BY Total_Revenue DESC LIMIT 2";
```

Select menu (1-8): 3

[Top 2 Brands by Revenue (Past 1 Year)]

Brand: Audi | Total Revenue: \$74000

Brand: Mercedes-Benz | Total Revenue: \$69000

Sale의 date 인덱스를 활용하여 최근 1년 이내의 판매 데이터만 인덱스 Range Scan으로 필터링하여 스캔 범위를 대폭 축소한다. 필터링된 매출 레코드를 기반으로 Vehicle, Model, Brand과 조인을 처리한 후, Brand.brand_id 및 Brand.name을 기준으로 그룹화한다. 이때 옵티마이저는 각 브랜드별 매출액(sale_price의 SUM)을 계산하여 메모리 내 임

시 테이블에 저장하고, 인덱스가 없는 집계 결과 영역에서 상위 2개 레코드만 고속 추출하기 위해 Top-N 소트(Sort) 알고리즘을 적용하여 내림차순 정렬 및 LIMIT 2 연산을 수행한다.

[query 4]

```
const char *query =
    "SELECT B.name, COUNT(S.sale_id) AS Unit_Sales FROM Sale S "
    "JOIN Vehicle V ON S.VIN = V.VIN JOIN Model M ON V.model_id = M.model_id "
    "JOIN Brand B ON M.brand_id = B.brand_id "
    "WHERE S.date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR) "
    "GROUP BY B.brand_id, B.name ORDER BY Unit_Sales DESC LIMIT 2";
```

Select menu (1-8): 4

[Top 2 Brands by Unit Sales (Past 1 Year)]

Brand: Honda | Total Units Sold: 1

Brand: Mercedes-Benz | Total Units Sold: 1

Query 3과 동일하게 Sale.date 인덱스를 타겟으로 지정하여 최근 1년 치의 유효 판매 레코드만 고속으로 추출한다. 동일한 릴레이션 조인 과정을 거쳐 브랜드별로 그룹화한 뒤, 이번에는 매출 금액 합산이 아닌 판매 대수(COUNT(S.sale_id))를 집계한다. 데이터베이스 엔진은 메모리 버퍼 상에서 이 집계 값을 기준으로 정렬을 수행하며, LIMIT 2 조건에 매칭되는 최상위 2개의 브랜드 레코드 가려내기가 완료되는 즉시 정렬 연산을 조기 종료하고 결과를 반환한다.

[query 5]

```
const char *query =
    "SELECT MONTH(S.date) AS Month, COUNT(S.sale_id) AS Total_Sales "
    "FROM Sale S JOIN Vehicle V ON S.VIN = V.VIN "
    "JOIN Model M ON V.model_id = M.model_id WHERE M.style = ? "
    "GROUP BY Month ORDER BY Total_Sales DESC LIMIT 1";
```

```
Select menu (1-8): 5
```

```
[Seasonal Sales Patterns]
```

```
Enter the vehicle style to search (e.g., Sedan, SUV, Convertible): Convertible  
The best-selling month for 'Convertible' is Month 8 with 1 units sold.
```

사용자가 콘솔에서 입력한 차량 외형 스타일(예: Sedan, SUV 등) 조건이 프리페어드 스테이트먼트를 통해 바인딩되어 Model.style 조건절에 대입된다. 엔진은 해당 스타일을 만족하는 Model을 인덱스 혹은 테이블 스캔을 통해 필터링한 뒤, 연쇄적으로 Vehicle과 Sale 테이블을 조인한다. 추출된 데이터셋의 판매 날짜에 MONTH() 함수를 적용하여 1월부터 12월까지의 월별 집계를 생성하며, COUNT(S.sale_id) 기준으로 정렬하여 가장 판매량이 높은 최상위 1개 월의 데이터만 Top-1 프로젝션을 통해 최종 결과로 도출한다.

[query 6]

```
const char *query =  
    "SELECT D.name, AVG(DATEDIFF(CURDATE(), V.manufacturing_date)) AS Avg_Days "  
    "FROM Vehicle V JOIN Dealer D ON V.dealer_id = D.dealer_id "  
    "WHERE V.cur_state = 'stock' GROUP BY D.dealer_id, D.name "  
    "ORDER BY Avg_Days DESC LIMIT 1";
```

```
Select menu (1-8): 6
```

```
[Dealer Inventory Efficiency (Worst Turnover)]
```

```
Dealer: Seoul Central Auto          | Average Inventory Age: 509.0 Days
```

Vehicle 테이블에서 현재 완제품 재고 상태를 나타내는 데이터인 cur_state = 'stock'인 튜플들만을 Selection 연산으로 분리해낸다. 해당 차량 레코드들과 Dealer 테이블을 dealer_id 참조를 통해 조인한 후, 각 딜러별로 데이터를 그룹화한다. 데이터베이스 엔진은 그룹 내에서 오늘 날짜(CURDATE())와 차량 제조일자(manufacturing_date)의 차이를 계산하는 DATEDIFF 연산 결과들의 AVG를 임시 테이블 상에 계산한다. 마지막으로 평균 재고 일수가 가장 큰 딜러 1곳을 선별하기 위해 내림차순 정렬 후 LIMIT 1 연산을 수행한다.

[query 7]

```
const char *query =
"SELECT Su.name, COUNT(DISTINCT M.model_id) AS Models_Covered "
"FROM Supplier Su JOIN Plant Pl ON Su.supplier_id = Pl.supplier_id "
"JOIN Part P ON Pl.plant_id = P.plant_id JOIN Vehicle_Part VP ON P.part_id = VP.part_id "
"JOIN Vehicle V ON VP.VIN = V.VIN JOIN Model M ON V.model_id = M.model_id "
"GROUP BY Su.supplier_id, Su.name ORDER BY Models_Covered DESC LIMIT 1";
```

Select menu (1-8): 7

[Supplier Coverage Analysis]

Supplier: Mobis | Supplies parts for 4 distinct models.

Supplier부터 Plant, Part, Vehicle_Part, Vehicle, Model로 총 6개의 테이블이 데이터베이스 내부에서 다중 조인된다. 옵티마이저는 조인 성능 최적화를 위해 각 테이블의 기본키와 외래키 구조를 분석하여 최적의 조인 순서를 결정한다. 이후 공급사별로 데이터를 그룹화한 뒤, 공급사가 커버하는 차량 모델의 다양성을 측정하기 위해 COUNT(DISTINCT M.model_id) 연산을 수행한다. 엔진은 내부적으로 중복 모델을 제거하기 위해 정렬 혹은 해시 기반의 Distinct 필터링을 적용하여 고유 모델 수를 집계하고, 최댓값을 가진 상위 1위 공급사 레코드를 추출한다.

[Business Rule Check]

```
INSERT INTO Sale (date, pay_method, sale_price, VIN, customer_id, dealer_id)
VALUES ('2026-06-03', 'Credit Card', 29000, 'VIN000000000000001', 2, 2);

DELETE FROM Model WHERE model_id = 1;

INSERT INTO Model (name, year, base_price, style, brand_id)
VALUES ('ErrorCar', 2026, -5000, 'Sedan', 1);
```

위의 3가지 쿼리는 각각 UNIQUE 조건, ON DELETE RESTRICT, CHECK 조건을 확인하기 위해 돌리는 쿼리이다. 이미 존재하는 VIN 추가, 아직 vehicle이 있는 상태에서의 Model 삭제, base_price의 양수 조건 체크를 테스트한다.

✖	51	13:56:34	INSERT INTO Sale (date, pay_method, sale_price, VIN, customer_id, dealer_id) VALUES ('2026-06-03', 'Credit Card', 29000, 'VIN000000000000001', 2, 2);	Error Code: 1062. Duplicate entry 'VIN000000000000001' for key 'sale.VIN'	0.015 sec
✖	52	13:56:49	DELETE FROM Model WHERE model_id = 1;	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint prevents this delete operation	0.031 sec
✖	53	13:57:01	INSERT INTO Model (name, year, base_price, style, brand_id) VALUES ('ErrorCar', 2026, -5000, 'Sedan', 1);	Error Code: 3819. Check constraint 'model_chk_1' is violated.	0.000 sec

3가지 case 모두 에러로 처리되는 것을 확인할 수 있다.